

Instructions Before You Open a PR

Applies to every real open-source organisation: Oppia, Rocket.Chat, Hacktoberfest repos, GSoC / LFX orgs and others.

Why this matters: Opening a PR for an issue you are not assigned to gets the PR closed. Repeating it can get you banned from the organisation, and maintainers remember names.

1. Ask first — the correct way

1. **Read the issue fully and reproduce the bug** on your own machine first.
2. **Comment on the issue asking to be assigned**, with what you found and your plan.
3. **Wait until a maintainer assigns you.** Do not write code or open a PR before that.
4. **If asked for proof, give it first** — a video or screenshots of the bug and of your fix.
5. **After assignment:** create a branch, fix it, and open the PR with **Fixes #<issue>** in the description.

2. Comment template

Hi, I'd like to work on this issue.
 I was able to reproduce it locally: <1-2 lines on what you saw>.
 My approach: <1-2 lines on how you plan to fix it>.
 Could you please assign it to me?

3. Never do this

Don't	Why
Open a PR without being assigned	PR gets closed; repeated cases lead to a ban
Work on an issue assigned to someone else	Wastes everyone's time and is seen as rude
Open two PRs for the same issue	Close the old one and update a single PR instead
Spam "please assign" or tag maintainers repeatedly	Maintainers are volunteers — wait 2-3 days before a polite follow-up
Submit AI-generated code you don't understand	You must explain every line in review

4. Before you pick an issue

- **Read all comments on the issue.** Check that nobody is already working on it and no PR is linked.
- **Skip issues labelled needs triage, not ready or discussion** — maintainers have not decided they should be worked on.
- **Take one issue at a time.** Claiming many issues at once ("issue hoarding") annoys maintainers.
- **Can't finish? Release it:** comment "I'm unable to continue, please unassign me". Never disappear silently.
- **Read CONTRIBUTING.md and the Code of Conduct first.** Every organisation has its own rules.

5. While making the PR

- **Never open a PR from your fork's main branch.** Create a new branch, and sync your fork with upstream first.
- **Keep the PR small and focused.** No reformatting whole files, no unrelated changes, no needless package-lock.json changes.

- **Run tests and lint locally before pushing.** CI checks on the PR must be green — fix red checks yourself.
- **Sign the CLA / DCO if the organisation requires it** (Oppia, Google, CNCF projects do). Without it the PR cannot be merged.
- **Commit with the email linked to your GitHub account**, or the contribution will not show on your profile.
- **Fill the PR template completely**, with Fixes #<issue>, and screenshots or a video for UI changes.

6. During review

- **Be patient.** Maintainers are often volunteers; reviews can take 3–7 days. Follow up politely, once.
- **Reply to every review comment** and push changes to the same PR. Do not close it and open a new one.
- **Don't DM maintainers.** Use the organisation's Slack / Zulip / Discord or GitHub Discussions, and ask in public channels.

7. Avoid spam and handle security properly

- **No low-value PRs:** typo-only fixes, adding your name to a README, whitespace changes. They get labelled spam / invalid and can get your account flagged in events like Hacktoberfest.
- **Found a security bug? Do not open a public issue.** Report it privately as described in the repo's SECURITY.md.

8. For yourself

- **Keep a professional GitHub profile:** real name, photo and a short bio.
- **Keep a list of all your issue and PR links** — you will need it for GSoC / LFX proposals and your resume.
- **Be respectful everywhere.** Maintainers remember contributors — good and bad.

Assignment 3 note: For ShopSphere / StayNest (Assignment 3), follow the rules in the Assignment 3 PDF. This guide is for real open-source organisations, where these rules are strictly enforced.

— Vikas Patel, Mentor & Project Manager